A Differentiable Atari VCS: A Complex, Fully Known Ground Truth for Explainable AI

Anonymous submission

Abstract

Explanation requires ground truth: to verify that an account of a system is correct, we must know the system's inner functioning. This is precisely what is missing wherever explainable AI (XAI) is most needed. The systems we can study today fall into two camps. Either they are simple and procedural— decision trees, rule lists, sparse linear models—where the mechanism is known but trivial, so explaining it tests nothing; or they are genuinely complex—deep networks, real-world tasks—where XAI is needed but no ground truth of the inner functioning exists, so an explanation can be plausible, confident, and wrong with no way to tell. We set out to remove this dichotomy by building the foundation for a study object that is at once genuinely complex and fully known—and, so that gradient-based methods can apply, fully differentiable. We re-implement the Atari 2600 Video Computer System (VCS)—a real computer architecture, and the platform on which deep reinforcement learning was born—as two independent, end-to-end *differentiable* emulators, one in Julia (JUTARI) and one in JAX (JAXTARI), each validated bit-for-bit against the XITARI reference. The Julia port reproduces XITARI exactly on all 64 supported Arcade-Learning-Environment games: 64/64 byte-identical RAM and 64/64 pixel-identical screens. Treating the cartridge ROM as a weight tensor, the RAM as a soft tape, and control flow as gates, we prove that the differentiable (soft) execution is bit-exact equal to the original (hard) execution in the forward pass at any finite temperature, while exposing gradients where bit logic has none. The system was built in roughly 137 hours of active work over 29 calendar days, with large parts written autonomously by coding agents; driving the re-implementation to pixel exactness even surfaced a latent non-determinism bug in the reference itself. This paper builds and validates the foundation; turning the XAI toolkit on it is the program it opens.

1 Introduction

When we explain a system—in explainable AI (XAI) as anywhere else—we are making a claim about its inner functioning: *this* part is responsible for *that* behaviour. To check whether such a claim is correct, we need to know the inner functioning independently, as ground truth. Without it we can ask whether an explanation is plausible, or whether humans find it convincing, but not whether it is *true*. Ground truth is the prerequisite for verification, and it is exactly what is missing where explanation matters most.

The systems available to study fall into two unsatisfying camps. On one side are simple, procedural models—decision trees, rule lists, sparse linear models—whose inner functioning is fully known. But here the explanation is essentially the model itself; explaining such a model tests an XAI method on nothing it could get wrong, and these are not the systems for which XAI was created. On the other side are the systems XAI actually targets—deep neural networks, learned agents, real-world pipelines—which are genuinely complex, and precisely for that reason have no available ground truth of their inner functioning. There, an attribution or saliency map can be plausible, confident, and wrong, and we have no way to tell (Atrey, Clary, and Jensen 2020; Nikulin et al. 2019). Known-but-trivial, or complex-but-unknown: there is no study object that is at once complex enough to be a real test and known well enough to grade the answer.

The closest anyone has come to escaping this dichotomy reached outside machine learning. In a now-famous study, Jonas and Kording (2017) asked whether the standard toolkit of systems neuroscience—connectomics, tuning curves, lesioning, dimensionality reduction—could recover how a MOS 6502 microprocessor computes, using the chip as a model organism precisely because its mechanism is complex *and* completely known. With perfect and unlimited data, the methods produced structure that looked meaningful but did not recover the processor's actual organisation; the conclusion was that the *methods* were lacking, and that the way to find out is to test them on a system whose ground truth we already possess. Two things, however, stop that study short of a substrate for modern XAI: the microprocessor was not *differentiable*, so the gradient-based methods at the centre of today's toolkit could not be applied to it; and the analyses run on it were classical statistics, not XAI. The MOS 6502 is no accident here—it is the CPU at the heart of the Atari 2600 Video Computer System (VCS), the very platform on which modern deep reinforcement learning was demonstrated (Mnih et al. 2013, 2015; Bellemare et al. 2013).

We therefore set out to build the missing quadrant: a study object that is simultaneously *complex* (a real computer architecture), *fully known* to us bit-for-bit, and *differentiable*, so

that the gradient-based XAI toolkit can in principle be turned on it. We re-implement the VCS—a 6507 CPU that fetches and executes code from a cartridge ROM, a Television Interface Adapter (TIA) that turns register writes into pixels cycle by cycle, and a RIOT chip providing RAM, a timer, and joystick and switch I/O—as two independent, end-to-end differentiable emulators: JUTARI in Julia (Bezanson et al. 2017) and JAXTARI in JAX (Bradbury et al. 2018). Each runs in a bit-exact *hard* mode and a differentiable *soft* mode, and each is validated against XITARI (DeepMind 2016), the Stella-derived C++ emulator (Stella Team 2024) that served as the reference for the original deep-Q-network (DQN) work.

This paper *builds and validates that foundation*. We deliberately do not yet run an XAI study on it; constructing a complex, fully known, differentiable substrate—and proving it faithful—is itself the contribution, and the XAI experiments it enables are the program it opens. Our contributions are:

- **Two differentiable VCS ports.** Independent Julia and JAX implementations of the full VCS (CPU, TIA, RIOT, cartridge bank-switching, console, controllers), each with a bit-exact hard path and a differentiable soft path. The Julia port is bit-for-bit identical to XITARI on all 64 supported games—64/64 byte-identical RAM and 64/64 pixel-identical screens.
- **A soft/hard formulation with a proof.** We treat the ROM as a weight tensor, the RAM as a soft tape, and control flow as gates, and we show that the soft forward pass is bit-exact equal to the hard forward pass at any finite temperature (Theorem 1), while gradients flow where bit logic provides none; we give the temperature-limit error bound for the fully relaxed variant (Theorem 2).
- **An AI-assisted engineering account.** A quantified report of how an emulator port that would classically cost many person-months was built in roughly 137 hours of active work, measured from the version-control log, with large parts written autonomously by coding agents.
- **A conformance evaluation** across the 64-game ALE set, with formally defined bit- and pixel-exactness metrics, and the finding that a bit-exact re-implementation is itself an audit tool for its reference: it exposed a non-determinism bug in XITARI.
- **A proof-of-concept gradient readout** showing that the foundation works as intended: exact register- and ROM-byte gradients that localise to the correct hardware element when checked against the known mechanism—a demonstration that the substrate is ready for XAI, not an XAI study in itself.

We are careful throughout to separate what we have *proven*, what we have *measured*, and what we *conjecture*. The differentiable VCS is the instrument; building it, and proving it faithful, is the work of this paper, and turning the full XAI toolkit on it—scoring each method against the known mechanism—is the program it opens rather than closes.

2 Background and Related Work

Explanation Needs Ground Truth

One response to the absence of ground truth is to give up generality and study only models that are transparent by construction: decision trees, rule lists, sparse linear models, or interpretable-by-design surrogates. For these the inner functioning is known—but it is known because it is trivial, and the explanation is little more than the model written out. Such models are not the ones XAI was built for, and reproducing their structure tests an XAI method on a problem it cannot fail. The interesting regime is the opposite one, and it is where ground truth disappears.

The dominant XAI tools are *attribution* or *saliency* methods, which assign each input element a number reflecting how much it mattered to the output. Grad-CAM computes the gradient of a target output with respect to a convolutional layer and renders it as a heatmap of important regions (Selvaraju et al. 2020); Grad-CAM++ refines this with a pixel-wise weighting of positive gradients (Chattopadhyay et al. 2018). These methods are powerful and architecture-agnostic, but they share a limitation their own authors flag: there is no way to confirm that a heatmap reflects the network's true reason for deciding, because for a deep network that reason is unknown. Grad-CAM++ notes explicitly that conventional faithfulness metrics "need not correlate with the actual factors responsible for the network's decision" (Chattopadhyay et al. 2018). Validation therefore falls back on proxies—bounding-box overlap, or human-trust studies.

The problem sharpens when the object of explanation is a reinforcement learning (RL) agent rather than an image classifier. The canonical such agent is the DQN, which learns to play Atari games from raw pixels (Mnih et al. 2013, 2015) and became the field's standard testbed. Greydanus et al. (2018) introduced a perturbation-based saliency for Atari agents and narrated strategies such as Breakout "tunneling"; tellingly, the one case in which they can claim the saliency is correct is an artificial experiment where they themselves planted the ground-truth feature. Nikulin et al. (2019) built saliency directly into the agent and validated it against human eye-tracking—a behavioural surrogate, not the machine's own computation. Atrey, Clary, and Jensen (2020) then made the consequence explicit: using counterfactual interventions (for example, mirroring the Breakout wall) they showed that popular saliency-based explanations of Atari agents are frequently unfalsifiable and do not hold up, and concluded that saliency maps are best treated as an *exploratory*, not an *explanatory*, tool. Such et al. (2019) industrialised the comparison of trained agents while leaving the validation gap untouched.

Surveys of explainable RL confirm that this is the field's structural condition rather than an incidental gap (Qing et al. 2022; Vouros 2023; Cheng, Yu, and Xing 2025; Saulières 2025). Across hundreds of methods the target is uniformly described as a closed or black box, and objective, mechanism-grounded evaluation is repeatedly named among the field's open needs. Even methods designed for interpretability inherit the problem: XDQN makes a DQN interpretable by training a transparent surrogate to mimic it, but its strongest

correctness measure is fidelity *to the DQN*—agreement with another black box, not with a known mechanism (Kontogiannis and Vouros 2023). The recurring obstacle is exactly the one Jonas and Kording (2017) identified: without a study object whose true mechanism is available, an explanation cannot be graded as right or wrong. We build such an object—complex, fully known, and differentiable—and prove it faithful; using it to grade XAI methods is the program it enables.

Known Operators and Differentiable Programming

Our approach is a relative of *known-operator learning*. Where a purely learned pipeline must relearn everything from data, Maier et al. (2019b) showed that embedding analytically known, differentiable operators directly into a network provably reduces its maximum error bound: if a layer is known, its error term vanishes and the corresponding part of the bound cancels, even for networks of arbitrary depth. The guiding principle is general—"any operation that allows computation of a gradient or sub-gradient towards its inputs" can be embedded (Maier et al. 2019b)—and the same spirit underlies physics-informed neural networks, which embed known governing equations (Raissi, Perdikaris, and Karniadakis 2019), and gentle, operator-aware treatments of deep learning for practitioners (Maier et al. 2019a). Our VCS is, in this sense, an extreme case: *every* operator is known, so the entire forward computation is a fixed, differentiable program, and the only thing left to "explain" is how that known program maps inputs to outputs.

Making a hand-written emulator differentiable requires a general-purpose differentiable-programming substrate. Julia provides source-to-source automatic differentiation (AD) over essentially arbitrary code via Zygote (Innes 2019), with eager execution and mutable state that map naturally onto an emulator's registers and RAM. JAX provides trace-based AD with just-in-time compilation to XLA (Bradbury et al. 2018), which favours pure, functional, array-shaped code. The two make different trade-offs for this workload, and building both lets us cross-check each port against the other in addition to the reference. We treat the cartridge ROM as a weight tensor, the 128 bytes of RAM as a Neural-Turing-Machine-style addressable tape (Graves, Wayne, and Danihelka 2014), and discrete control flow as gates relaxed with the Gumbel-softmax reparameterisation (Jang, Gu, and Poole 2017) and straight-through estimators (Bengio, Léonard, and Courville 2013).

Why the Atari VCS Is Good Ground Truth

The VCS (Figure 1) is a real computer architecture, small enough to know completely yet complex enough to be interesting. A 6507 CPU (a cost-reduced MOS 6502) executes a program stored in the cartridge ROM. Because the address space is only 13 bits wide, larger cartridges *bank-switch*: writes to special addresses swap which 4 KB ROM page is visible. The RIOT chip (M6532) holds the 128 bytes of system RAM, an interval timer, and the input ports for the joystick, paddles, and console switches. The heart of the machine is the TIA: it has no frame buffer, so the CPU must race the electron beam, writing colour, sprite, and playfield registers in time with each scanline. The screen and audio are the

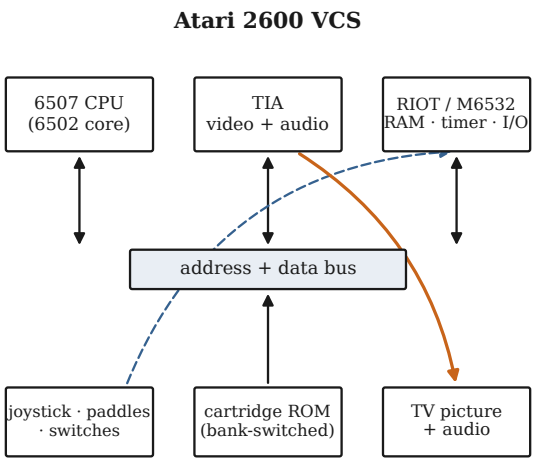

Figure 1: The Atari 2600 VCS. A 6507 CPU executes code from a (bank-switched) cartridge ROM over a shared address/data bus; the RIOT provides 128 bytes of RAM, a timer, and joystick/switch I/O; the TIA turns register writes into a picture and sound, cycle by cycle. Every block is re-implemented and differentiated in both ports.

TIA's outputs. This is the system the Arcade Learning Environment (Bellemare et al. 2013) exposes to RL agents, and that XITARI (DeepMind 2016), a fork of the Stella emulator (Stella Team 2024), simulates as the reference for DQN. Because XITARI is deterministic given a ROM, an action stream, and a seed, it is an ideal oracle: every register, RAM byte, and pixel our ports produce can be compared against it, and any disagreement is unambiguously a port bug—or, as we found, a bug in the reference.

3 Building Two Differentiable VCS Ports

Reference-Driven, Conformance-Gated Engineering

We ported XITARI subsystem by subsystem—CPU, bus, TIA, RIOT, cartridge mappers, console, controllers, and the ALE-style environment wrapper—under a strict conformance gate. Three harnesses define "correct." **PXC1** compares each port against a per-instruction XITARI trace (registers, RAM, and TIA state). **PXC2** cross-checks the two ports against each other, so a bug present in both the reference and one port can still be caught by the second, independent implementation. **PXC-S** compares the full 210×160 frame buffer. Each port implements all 151 documented NMOS 6502 instructions plus the undocumented `USBC` alias and 37 common "illegal" opcodes (NOP/LAX/SAX families)—189 opcode bytes in total—in both execution modes, and nine cartridge mappers (2K, 4K, F8, F6, F4, their Superchip variants, and E0) with content-based auto-detection mirroring the reference. Table 1 summarises the two ports.

This is work that would classically be measured in person-months of expert labour. The cycle-level timing of the TIA, the sub-cycle behaviour of the RIOT timer, and the boot-time

	JUTARI	JAXTARI
Language / AD	Julia / Zygote	JAX / XLA
Source lines	8,427	9,464
Test lines	6,052	11,788
Test cases	∼1,187	803
Opcode bytes	189	189
Cartridge mappers	9	9
RAM-exact (of 64)	**64**	live PXC
Pixel-exact (of 64)	**64**	catch-up
Soft throughput (steps/s)	∼363,800	∼1,666

Table 1: The two differentiable ports at a glance. JUTARI is the conformance lead and is bit-for-bit identical to XITARI on all 64 games; JAXTARI mirrors it under the live PXC harnesses and trails on the most recent boot/render fixes (it is ∼200× slower per frame, so it never blocks a JUTARI deliverable).

quirks of bank-switched cartridges are exactly the kind of detail that resists a clean specification and must be recovered by tracing the reference. We carried it out with coding agents operating largely autonomously under the conformance gates, committing after each step so that the version-control log doubles as a developer journal. We quantify the effort in the Results.

Hard and Soft Execution

Each port runs in two modes that share one code path (Figure 2). In **hard** mode, the emulator is the ordinary bit-exact machine: opcode bytes index an integer dispatch table, the ALU is integer arithmetic, and flags are bits. In **soft** mode, the same step is re-expressed so that gradients can flow. Three relaxations carry the idea.

First, every ROM and RAM read is written as a dot product against a one-hot address vector. For a ROM tensor $r \in \mathbb{R}^N$ and address $a \in \{0, \dots, N-1\}$,

$$\mathrm{peek}(r, a) \;=\; \mathbf{1}_a^\top r \;=\; \sum_i [\![i = a]\!]\, r_i \;=\; r_a. \qquad (1)$$

The forward value is exactly r_a, but the gradient $\partial\,\mathrm{peek}/\partial r = \mathbf{1}_a$ is now defined and one-hot—this is what makes "which ROM byte explains this pixel?" a gradient question. The RAM tape uses the identical construction, which is the discrete limit of the soft, attention-style addressing of a Neural Turing Machine (Graves, Wayne, and Danihelka 2014).

Second, opcode dispatch is, in principle, a convex combination over the handler outputs. With logits $\ell \in \mathbb{R}^K$ (here $K = 256$) and a temperature $T > 0$,

$$w_k = \mathrm{softmax}(\ell/T)_k = \frac{e^{\ell_k/T}}{\sum_j e^{\ell_j/T}}, \qquad (2)$$
$$\mathrm{select}(\ell, V; T) = w^\top V,$$

which collapses to the hard pick as $T \to 0$ (Jang, Gu, and Poole 2017). In the executed step we use the saturated form directly—a hard switch on the decoded opcode—so the forward pass is exact; the relaxed form (2) is what a fully soft variant would use.

Third, a conditional branch "if flag then $\mathrm{pc} \mathrel{+}= \delta$" is relaxed with a sigmoid gate. Writing the relevant float-valued flag $f \in [0, 1]$ as a logit $z = s\,(2f - 1)$ (with sign s for branch-when-set or -clear) and sharpness α,

$$g = \sigma(\alpha z), \qquad \mathrm{pc}_{\mathrm{soft}} = (1 - g)\,\mathrm{pc}_{\bar{b}} + g\,\mathrm{pc}_b. \qquad (3)$$

As $\alpha \to \infty$ the gate saturates to a hard branch.

These relaxations are glued to the exact machine by a *straight-through estimator* (STE) (Bengio, Léonard, and Courville 2013). For any soft quantity with a matching hard value,

$$\mathrm{STE}(\mathrm{soft}, \mathrm{hard}) = \mathrm{soft} + \mathrm{sg}(\mathrm{hard} - \mathrm{soft}), \qquad (4)$$

where $\mathrm{sg}(\cdot)$ is the stop-gradient operator. The forward value is exactly *hard*; the backward pass sees $\partial(\mathrm{soft})$. The round and clamp operations are treated the same way (forward exact, backward identity inside the valid range). The consequence, made precise next, is that soft mode reproduces the hard machine *exactly* in the forward pass and differs only in the gradient it exposes.

This is the same device that makes *max-pooling* differentiable, and it applies far beyond branches. Every hard decision in the machine is a discrete *selection*: which opcode handler runs, whether a branch is taken, which object owns a pixel under TIA priority, which sprite bit lights it. Each is a switch whose winner we record in the forward pass; the backward pass then routes the gradient to the selected value, exactly as max-pooling routes its gradient to the argmax input. With the switch stored, the entire *content* path—register values, sprite colours and graphics bits, priority compositing—is differentiable with no approximation and a bit-exact forward pass. The one quantity a stored switch cannot differentiate is a discrete *index*: the column at which a sprite is placed by strobe timing, for the same reason max-pooling gives no gradient with respect to the pooling location. There, and only there, a differentiable sampler—a sub-pixel triangular (bilinear) kernel, as in spatial transformer networks (Jaderberg et al. 2015)—restores the position gradient. We use this distinction in the qualitative study below.

4 Soft Equals Hard: Equivalence and Gradients

We now state what the construction guarantees. Let Φ_{H} and Φ_{S} be the one-step transition maps of the hard and soft emulators, acting on the full state x (CPU registers, RAM, RIOT, TIA, and frame buffer). The first result is that the two maps agree in value.

Theorem 1 (Exact forward equivalence)**.** *For every reachable state x and every finite sharpness α and temperature T, the soft and hard one-step maps coincide in float32, $\Phi_{\mathrm{S}}(x) = \Phi_{\mathrm{H}}(x)$. Consequently the H-step trajectories coincide, $x_t^{\mathrm{S}} = x_t^{\mathrm{H}}$ for all $t \le H$. The modes differ only in the Jacobian $\partial\Phi_{\mathrm{S}}/\partial x$, which is defined and generically nonzero where $\partial\Phi_{\mathrm{H}}/\partial x$ is zero or undefined.*

Proof sketch. Each handler's forward output is a composition of (i) one-hot ROM/RAM reads, which equal ordinary indexing by (1); (ii) integer/round arithmetic for registers, flags, and timers; (iii) a hard dispatch on the decoded opcode; and (iv) the branch, whose forward value

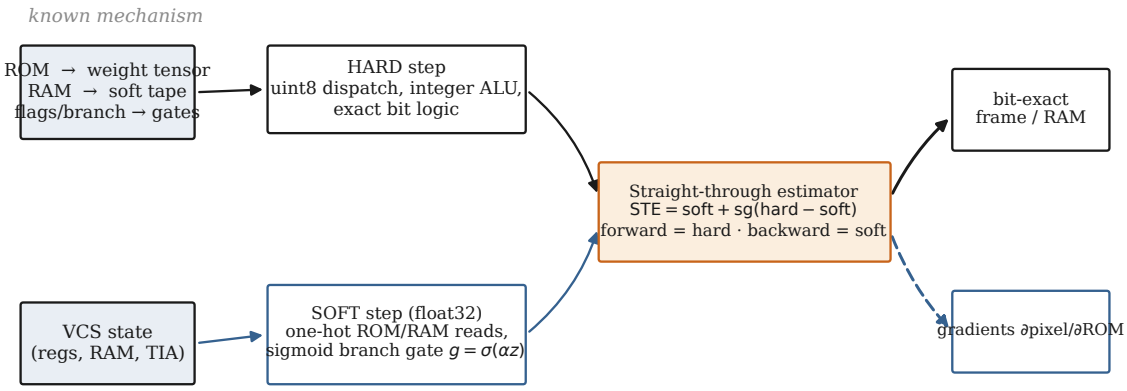

Figure 2: The two execution paths share one known mechanism. The HARD step uses integer dispatch and exact bit logic; the SOFT step reads the ROM and RAM as one-hot dot products and relaxes the branch decision with a sigmoid gate. A straight-through estimator joins them: the forward value is the hard one (bit-exact), while the backward pass uses the soft gradient, so the substrate emits both an exact frame and exact gradients of pixels with respect to ROM bytes and registers.

is $\mathrm{STE}(\mathrm{pc}_{\mathrm{soft}}, \mathrm{pc}_{\mathrm{hard}}) = \mathrm{pc}_{\mathrm{hard}}$ by (4). Each component is forward-identical to its hard counterpart, and the hard counterpart is validated bit-for-bit against the reference (the PXC harnesses). The stop-gradient in (4) alters only the backward rule, not the value. Since small integers up to 2^{24} are represented exactly in float32, the equality is bit-exact. □

Theorem 1 is the strong, honest claim: attribution computed in soft mode is computed on the *true* hard trajectory, not on an approximation of it. It also makes a temperature sweep of forward error vacuous—the error is identically zero—so the interesting question is the behaviour of a *fully* relaxed variant that does not apply the straight-through correction. For that we need a mild non-degeneracy assumption.

Assumption 1 (Decision margins)**.** *Along the executed trajectory of length H, every discrete decision has a strictly positive margin. For opcode dispatch at step t with logits $\ell^{(t)}$ and winner $k_t^\star$, the logit gap $\Delta_t = \ell^{(t)}_{k_t^\star} - \max_{k \neq k_t^\star} \ell^{(t)}_k$ is positive; for each conditional branch with flag logit z_t, the margin $m_t = |z_t|$ is positive. Let $\Delta_{\min} = \min_t \Delta_t > 0$ and $m_{\min} = \min_t m_t > 0$.*

In the executed dispatch the flags are exact bits, so $z_t = \pm 1$ and $m_{\min} = 1$; Assumption 1 holds trivially for branches and is a statement only about ties in a fully soft dispatch.

Theorem 2 (Temperature-limit bound)**.** *Consider the fully relaxed emulator that uses the softmax dispatch* (2) *and sigmoid branch* (3) without *the straight-through correction. Under Assumption 1, its one-step map converges to the hard map as $T \to 0$ and $\alpha \to \infty$, with per-step deviation*

$$\left\|\Phi_{\mathrm{S}}^{T}(x) - \Phi_{\mathrm{H}}(x)\right\|_{\infty} \;\le\; C\Big[(K-1)\, e^{-\Delta_{\min}/T} + e^{-\alpha m_{\min}}\Big], \tag{5}$$

where C bounds the value range (bytes ≤ 255, branch offsets ≤ 256). Over a horizon H, with Lipschitz constant $L \ge 1$ for error propagation, the trajectory deviation is at most $\frac{L^H-1}{L-1}\, C[(K-1)e^{-\Delta_{\min}/T} + e^{-\alpha m_{\min}}] = \mathcal{O}(e^{-c/T})$ with $c = \Delta_{\min}$.

Proof sketch. The non-winning softmax mass is $1 - \mathrm{softmax}(\ell/T)_{k^\star} \le (K-1)e^{-\Delta_{\min}/T}$, so the convex combination (2) deviates from the winning row by at most $(K-1)e^{-\Delta_{\min}/T} \cdot C$. For the gate, $|\sigma(\alpha z) - [\![z > 0]\!]| = \sigma(-\alpha|z|) \le e^{-\alpha m_{\min}}$, so the blended program counter deviates by at most $C\, e^{-\alpha m_{\min}}$. Composing H steps gives the geometric accumulation. □

Corollary 1. *The straight-through estimator* (4) *attains the $T \to 0$ forward limit of Theorem 2* exactly at finite temperature*: it sets the bracket in* (5) *to zero for all T, α while keeping the same backward Jacobian as the limit. Theorem 1 is thus the $T \to 0$ forward limit, achieved by construction.*

These results are matched by the implementation's unit tests, which confirm that the softmax dispatch saturates to the exact hard pick at large logits, that the sigmoid branch saturates to the exact taken/not-taken program counter at large α, and that the soft memory read collapses to ordinary indexing at small T—the empirical face of Theorems 1–2.

5 Evaluation Methodology

We report five families of measurements, defined here so the results are unambiguous.

Bit-exactness in memory. For each game we run the standard ALE boot (60 no-op frames plus four resets) and then a fixed action stream, and compare the 128-byte RIOT RAM against XITARI after every frame. A game is *RAM-exact* if every byte of every frame is identical. We report the count of RAM-exact games out of 64.

Pixel-exactness on screen. Identically, we compare the full 210×160 frame buffer (palette indices) after every frame. A game is *pixel-exact* if every pixel of every frame is identical; we report the count out of 64.

Implementation wall-time. We estimate *active* implementation time from the version-control log. Treating any gap between consecutive commits longer than three hours as a session boundary, we sum the durations of the active sessions; we report the two- and four-hour thresholds as a sensitivity check.

Tests and bugs. We count authored test cases per port and the distinct numbered bug investigations recorded in the development log.

Throughput. We measure soft-mode steps per second on a fixed ROM after just-in-time warm-up, on a single Apple M1 Max core (JAX CPU backend; Julia 1.10), reporting the median of repeated runs.

6 Results

Conformance

The Julia port JUTARI is bit-for-bit identical to XITARI on *all* 64 supported ALE games: 64/64 games are RAM-exact (zero differing bytes across every frame) and 64/64 are pixel-exact (zero differing pixels across every frame). Reaching full pixel exactness required porting XITARI's actual per-colour-clock TIA object model rather than approximating its output—deferred sprite repositioning, the reset-then-skip-first-copy rule, the vertical-delay shadow latch, and the playfield-reflect gate—following the principle that the mechanism, not the scoreboard, is what must match, because the eventual XAI attribution projects gradients onto this very frame buffer. JAXTARI reproduces the same logic under the live PXC1/PXC2 harnesses and is being brought up to the most recent boot and render fixes; because it is roughly $200\times$ slower per frame, it is sequenced behind JUTARI and never blocks a deliverable.

Effort: Person-Months into Days

The project comprises 373 commits over a 29-day calendar span (Figure 3). Removing idle gaps longer than three hours leaves 52 active sessions totalling **136.8 hours**, or about **5.7 working days** of active implementation; the estimate is stable under the threshold choice (106 hours at a two-hour cutoff, 162 hours at four hours), and the median in-session cadence is one commit every $\sim$13 minutes. Roughly 80% of the calendar span was idle and excluded. Against this we built two independent emulators totalling $\sim$17,900 lines of source and a comparable volume of tests ($\sim$1,990 test cases across the two ports), and chased 67 distinct numbered bug investigations to a bit-exact result.

We state plainly that an estimate of the counterfactual—how long this would have taken without coding agents—is a *conjecture*, not a measurement. A faithful, cycle-accurate emulator of a system as quirky as the VCS, written and validated bit-for-bit against a reference, is conventionally a multi-person-month undertaking; comparable open-source emulators represent years of cumulative community effort. Producing two such ports, in two languages, plus a differentiable layer and a 64-game conformance suite, in under six working days of active effort represents a compression of one to two orders of magnitude in calendar-normalised labour. We offer this as an honest order-of-magnitude claim, anchored to the measured 137 hours, not as a controlled comparison.

Throughput

On the soft-mode benchmark, JUTARI sustains $\sim$363,800 steps per second and JAXTARI $\sim$1,666, a $\sim 218\times$ gap (Table 1). The difference is dominated by per-operation kernel-launch overhead in JAX: each soft step decodes one opcode and dispatches through a 256-way switch, and at single-instruction granularity the launch overhead dwarfs the work, whereas Julia inlines the small handlers. This is a property of the execution granularity, not of the AD systems: for a gradient-based XAI workflow, which takes one forward-plus-backward pass per attribution, JAXTARI's $\sim$30 s per 50,000-step trace is comfortably interactive and amortises over many attributions; for batched, training-scale rollouts the JAX path would instead want vectorised parallel environments.

Proof of Concept: Ground-Truth Gradients

The point of the foundation is that gradients computed on it can be *checked*. Because soft mode is forward-exact (Theorem 1), it yields exact gradients of any read-out with respect to the state and the ROM. We give a small qualitative study (Figure 4); it is a proof of concept that the substrate behaves as intended, not an XAI evaluation.

First, a Grad-CAM analogue. We differentiate the rendered frame with respect to the TIA colour registers: $\partial\text{screen}/\partial\texttt{COLUP0}$ localises exactly to the player-0 sprite, $\partial\text{screen}/\partial\texttt{COLUP1}$ to a second (distractor) sprite, and $\partial\text{screen}/\partial\texttt{COLUBK}$ to the background—each pixel attributing to the exactly correct register, a statement that can be *checked* here and cannot be on a black box. The stored-switch view extends this: a single sprite *graphics* bit, once represented as a value behind its switch, has a non-zero gradient to precisely the pixels it lights.

Second, the joystick. A naive gradient from the joystick to the screen is zero, because the path runs through a discrete sprite *position* index; this is not a failure of differentiation but the ground truth telling us that a naive gradient saliency would wrongly report "the joystick does not affect the screen." Applying the differentiable sampler to the position makes the relationship visible: the gradient of the on-screen sprite position with respect to the four joystick directions correctly recovers the control mapping (push UP+RIGHT to move the sprite toward a goal in the upper-right), and the per-direction $\partial\text{screen}/\partial\text{joystick}$ map highlights the sprite edges that move. We can *verify* this against the known wiring—which is the whole point.

Integrated gradients (Sundararajan, Taly, and Yan 2017) over ROM bytes is the natural next step; our harness computes it on short traces, and a full game-length ROM-attribution study is in progress (some long traces still hit an initialisation busy-loop in the soft RIOT timer, which we flag as a known limitation). Systematic XAI evaluation—scoring each method against the known mechanism—is the program this paper enables.

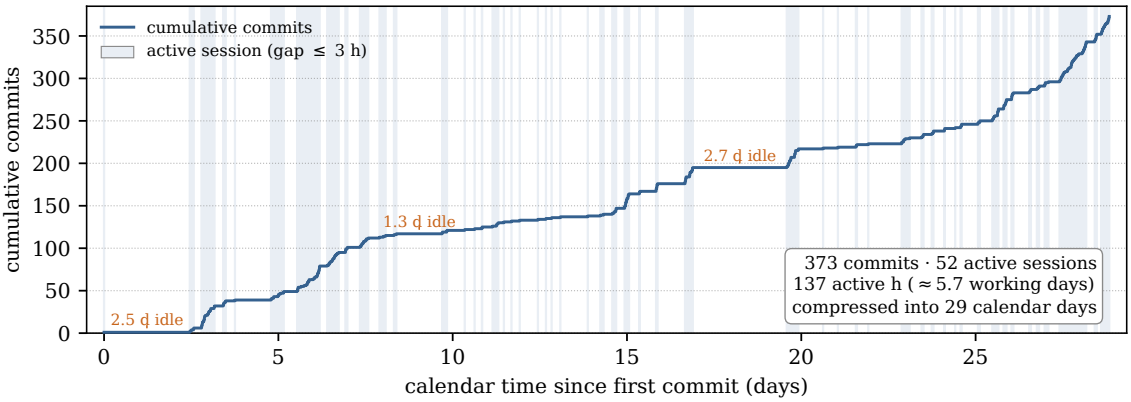

Figure 3: Implementation effort from the version-control log. Cumulative commits against calendar time; shaded bands are active sessions (consecutive commits no more than three hours apart). The long flat stretches are idle gaps. The 373 commits cluster into 52 active sessions totalling ∼137 hours of work—about 5.7 working days—spread across a 29-day calendar window.

7 Discussion

A new paradigm: build the instrument, then study it. The effort result points beyond this paper. When a faithful, differentiable re-implementation of a complex system can be produced in days rather than months, "build a known-ground-truth substrate and then study it with XAI" becomes a practical research method rather than a thought experiment. We believe—and offer this as a hypothesis—that the same recipe applies wherever a reference implementation exists and can serve as a conformance oracle: other consoles, instruction-set simulators, or hand-written numerical kernels.

A re-implementation audits its reference. Driving JUTARI to 64/64 pixel exactness surfaced a latent non-determinism bug in XITARI itself, the de-facto reference. One title's attract-mode demo reads *uninitialised* on-cartridge Superchip RAM as a cheap random-number source. XITARI seeds that RAM from `time(NULL)` and ignores its own seed parameter, so the title renders differently on every run despite a pinned seed—a divergence invisible to RAM-state checks and visible only on screen. We reach full conformance by pinning the seed deterministically in both emulators, which also makes the whole suite reproducible. The general lesson is methodological: an independent bit-exact re-implementation is not only validated *by* its reference, it is a tool for *auditing* that reference's reproducibility. We have prepared a fix to report upstream.

Notes from the build log. The hardest divergences were, unsurprisingly, in timing: the RIOT timer's reset state, the TIA's sub-cycle object rendering, and the boot-time format probe that seeds free-running counters games never re-initialise. Two model anecdotes are worth recording for the history. The bulk of the work (366 commits) was carried out by one frontier coding model; a single autonomous session was driven by a second model during an author absence; and the very first scaffolding was attempted by a third model that then declined the full port, unable to decompose it into modules and tests. We mention these only to document that autonomous capability on a task of this size is real but uneven across systems, and that the conformance gate—not the model—was what made the autonomy safe.

Limitations. JAXTARI is not yet at 64/64 on the most recent fixes; the full ROM-byte integrated-gradient study is incomplete; and the effort comparison is an order-of-magnitude estimate, not a controlled trial. None of these affects the core claims: the proven forward equivalence, and the measured 64/64 conformance of JUTARI.

8 Conclusion

We set out to give explainable AI something it has never had: a study object that escapes the known-but-trivial / complex-but-unknown dichotomy by being a genuine, complex information-processing system that is at the same time fully known and fully differentiable. We built it twice, in Julia and JAX, validated one port bit-for-bit against the reference on all 64 supported Atari games, and proved that the differentiable forward pass is bit-exact equal to the original machine while exposing gradients where bit logic has none. This is the foundation, not the experiment. With the mechanism fully known and differentiable, an attribution a method produces can now—for the first time on a system of this complexity—be scored as right or wrong against the truth. Turning the full XAI toolkit on this foundation, game by game and method by method, and so testing the methods themselves the way Jonas and Kording (2017) argued one should, is the program this paper makes possible.

Ethical Statement

This work involves no human subjects, personal data, or sensitive information. It uses Atari game ROMs solely as fixed computational inputs for conformance testing. The agentic software-engineering methodology we report is dual-use—faster construction of complex software can be applied well or poorly—and we mitigate this by coupling autonomy to a strict, automatically checked conformance oracle, which we

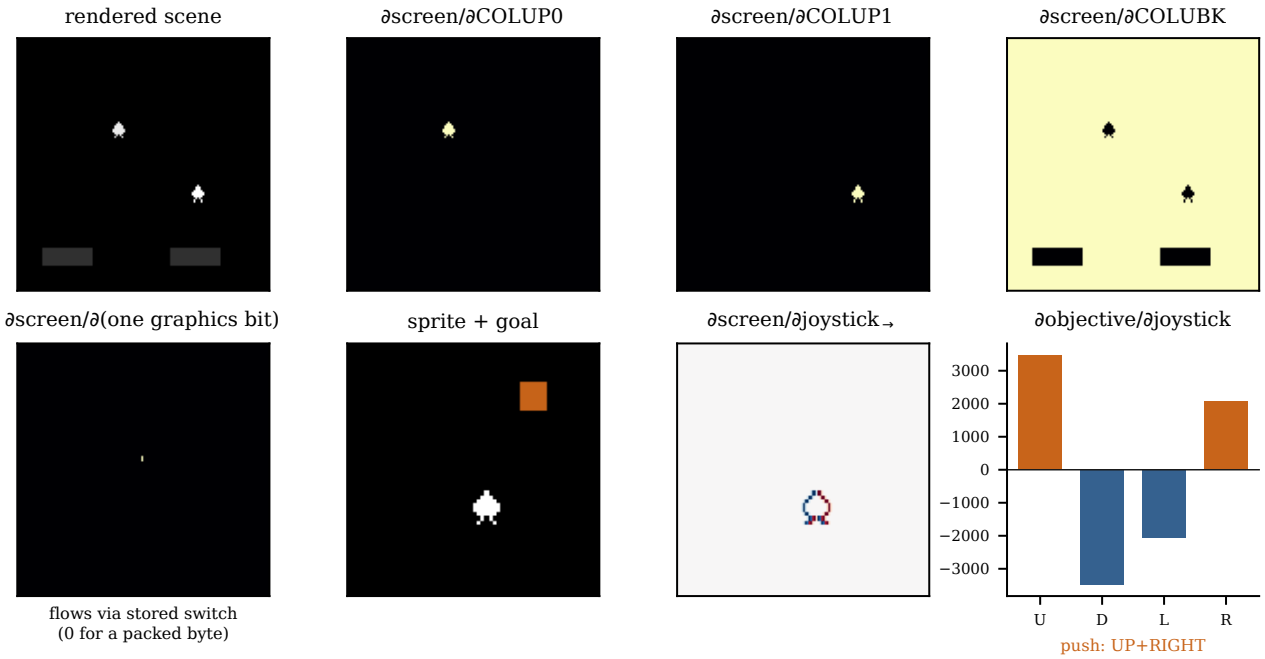

Figure 4: Qualitative XAI on the differentiable VCS, all computed by reverse-mode AD through JUTARI. **Top:** from the unmodified soft renderer, a Grad-CAM analogue—∂screen/∂register localises each object to exactly the pixels its register controls (player 0, the distractor player 1, and the background), a localisation that can be *checked* against ground truth. **Bottom, left to right:** a single sprite graphics bit attributes to its own pixels once represented as a value behind a stored switch (it is zero when read as a packed byte); the scene with a goal marker; the signed ∂screen/∂joystick saliency (a soft sampler makes the position differentiable), highlighting the sprite edges that move; and the gradient of the on-screen sprite position w.r.t. the four joystick directions, which correctly infers that pushing UP+RIGHT moves the sprite toward the goal.

view as the responsible pattern for such tools. Our intended use, validating XAI methods against known ground truth, is a contribution to the transparency and accountability of AI systems.

References

Atrey, A.; Clary, K.; and Jensen, D. 2020. Exploratory Not Explanatory: Counterfactual Analysis of Saliency Maps for Deep Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.

Bellemare, M. G.; Naddaf, Y.; Veness, J.; and Bowling, M. 2013. The Arcade Learning Environment: An Evaluation Platform for General Agents. *Journal of Artificial Intelligence Research*, 47: 253–279.

Bengio, Y.; Léonard, N.; and Courville, A. 2013. Estimating or Propagating Gradients through Stochastic Neurons for Conditional Computation. arXiv:1308.3432.

Bezanson, J.; Edelman, A.; Karpinski, S.; and Shah, V. B. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Review*, 59(1): 65–98.

Bradbury, J.; Frostig, R.; Hawkins, P.; Johnson, M. J.; Leary, C.; Maclaurin, D.; Necula, G.; Paszke, A.; VanderPlas, J.; Wanderman-Milne, S.; and Zhang, Q. 2018. JAX: Composable Transformations of Python+NumPy Programs. http://github.com/google/jax. Software.

Chattopadhyay, A.; Sarkar, A.; Howlader, P.; and Balasubramanian, V. N. 2018. Grad-CAM++: Generalized Gradient-Based Visual Explanations for Deep Convolutional Networks. In *IEEE Winter Conference on Applications of Computer Vision (WACV)*, 839–847. IEEE.

Cheng, Z.; Yu, J.; and Xing, X. 2025. A Survey on Explainable Deep Reinforcement Learning. *arXiv preprint arXiv:2502.06869*.

DeepMind. 2016. Xitari: An Arcade Learning Environment Fork. https://github.com/google-deepmind/xitari. Accessed: 2026-06-16.

Graves, A.; Wayne, G.; and Danihelka, I. 2014. Neural Turing Machines. arXiv:1410.5401.

Greydanus, S.; Koul, A.; Dodge, J.; and Fern, A. 2018. Visualizing and Understanding Atari Agents. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80, 1792–1801. PMLR.

Innes, M. 2019. Don't Unroll Adjoint: Differentiating SSA-Form Programs. arXiv:1810.07951.

Jaderberg, M.; Simonyan, K.; Zisserman, A.; and Kavukcuoglu, K. 2015. Spatial Transformer Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28.

Jang, E.; Gu, S.; and Poole, B. 2017. Categorical Reparam-

eterization with Gumbel-Softmax. In *International Conference on Learning Representations (ICLR)*.

Jonas, E.; and Kording, K. P. 2017. Could a Neuroscientist Understand a Microprocessor? *PLOS Computational Biology*, 13(1): e1005268.

Kontogiannis, A.; and Vouros, G. A. 2023. XDQN: Inherently Interpretable DQN through Mimicking. arXiv:2301.03043.

Maier, A.; Syben, C.; Lasser, T.; and Riess, C. 2019a. A Gentle Introduction to Deep Learning in Medical Image Processing. *Zeitschrift für Medizinische Physik*, 29(2): 86–101.

Maier, A. K.; Syben, C.; Stimpel, B.; Würfl, T.; Hoffmann, M.; Schebesch, F.; Fu, W.; Mill, L.; Kling, L.; and Christiansen, S. 2019b. Learning with Known Operators Reduces Maximum Error Bounds. *Nature Machine Intelligence*, 1(8): 373–380.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Graves, A.; Antonoglou, I.; Wierstra, D.; and Riedmiller, M. 2013. Playing Atari with Deep Reinforcement Learning. *arXiv preprint arXiv:1312.5602*.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; Petersen, S.; Beattie, C.; Sadik, A.; Antonoglou, I.; King, H.; Kumaran, D.; Wierstra, D.; Legg, S.; and Hassabis, D. 2015. Human-Level Control through Deep Reinforcement Learning. *Nature*, 518(7540): 529–533.

Nikulin, D.; Ianina, A.; Aliev, V.; and Nikolenko, S. 2019. Free-Lunch Saliency via Attention in Atari Agents. arXiv:1908.02511.

Qing, Y.; Liu, S.; Song, J.; Zhou, Y.; Chen, K.; Wang, H.; and Song, M. 2022. A Survey on Explainable Reinforcement Learning: Concepts, Algorithms, and Challenges. *arXiv preprint arXiv:2211.06665*.

Raissi, M.; Perdikaris, P.; and Karniadakis, G. E. 2019. Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations. *Journal of Computational Physics*, 378: 686–707.

Saulières, L. 2025. A Survey of Explainable Reinforcement Learning: Targets, Methods and Needs. *arXiv preprint arXiv:2507.12599*.

Selvaraju, R. R.; Cogswell, M.; Das, A.; Vedantam, R.; Parikh, D.; and Batra, D. 2020. Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *International Journal of Computer Vision*, 128(2): 336–359. Originally arXiv:1610.02391 (2016).

Stella Team. 2024. Stella: A Multi-Platform Atari 2600 VCS Emulator. https://stella-emu.github.io. Accessed: 2026-06-16.

Such, F. P.; Madhavan, V.; Liu, R.; Wang, R.; Castro, P. S.; Li, Y.; Zhi, J.; Schubert, L.; Bellemare, M. G.; Clune, J.; and Lehman, J. 2019. An Atari Model Zoo for Analyzing, Visualizing, and Comparing Deep Reinforcement Learning Agents. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, 3260–3267.

Sundararajan, M.; Taly, A.; and Yan, Q. 2017. Axiomatic Attribution for Deep Networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70, 3319–3328. PMLR.

Vouros, G. A. 2023. Explainable Deep Reinforcement Learning: State of the Art and Challenges. *ACM Computing Surveys*, 55(5): 1–39.